

5 full-stack mini projects with UI and backend code

1) To-Do List App

A to-do app is the best starter project because it teaches CRUD, state handling, and API calls in a very small codebase. It is also one of the most common beginners MERN projects.

Backend: server.js

js

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
let todos = [
```

```
  { id: 1, text: "Learn React", done: false },
```

```
  { id: 2, text: "Build API", done: false }]
```

```
];
```

```
app.get("/api/todos", (req, res) => res.json(todos));
```

```
app.post("/api/todos", (req, res) => {
```

```
  const { text } = req.body;
```

```
  if (!text || !text.trim()) return res.status(400).json({ message: "Text required" });
```

```
  const todo = { id: Date.now(), text: text.trim(), done: false };
```

```
  todos.push(todo);
```

```
  res.status(201).json(todo);
```

```
});
```

BrainStack

```
app.put("/api/todos/:id", (req, res) => {  
  const id = Number(req.params.id);  
  todos = todos.map(t => t.id === id ? { ...t, ...req.body } : t);  
  res.json({ message: "Updated" });  
});  
  
app.delete("/api/todos/:id", (req, res) => {  
  const id = Number(req.params.id);  
  todos = todos.filter(t => t.id !== id);  
  res.json({ message: "Deleted" });  
});  
  
app.listen(5000, () => console.log("Server running on 5000"));
```

Frontend: App.jsx

```
jsx  
  
import { useEffect, useState } from "react";  
import "./App.css";  
  
const API = "http://localhost:5000/api/todos";  
  
export default function App() {  
  const [todos, setTodos] = useState([]);  
  const [text, setText] = useState("");  
  
  const loadTodos = async () => {  
    const res = await fetch(API);  
    setTodos(await res.json());  
  };  
}
```

```
useEffect(() => {  
  loadTodos();  
}, []);
```

```
const addTodo = async (e) => {  
  e.preventDefault();  
  if (!text.trim()) return;  
  await fetch(API, {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ text })  
  });  
  setText("");  
  loadTodos();  
};
```

```
const toggleTodo = async (todo) => {  
  await fetch(`${API}/${todo.id}`, {  
    method: "PUT",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ done: !todo.done })  
  });  
  loadTodos();  
};
```

```
const deleteTodo = async (id) => {  
  await fetch(`${API}/${id}`, { method: "DELETE" });  
  loadTodos();  
};
```

```
return (  
  <div className="container">  
    <h1>To-Do List</h1>  
    <form onSubmit={addTodo}>  
      <input value={text} onChange={(e) => setText(e.target.value)} placeholder="Add task"  
    />  
      <button type="submit">Add</button>  
    </form>  
    <ul>  
      {todos.map((todo) => (  
        <li key={todo.id}>  
          <span onClick={() => toggleTodo(todo)} style={{ textDecoration: todo.done ? "line-through" : "none" }}>  
            {todo.text}  
          </span>  
          <button onClick={() => deleteTodo(todo.id)}>Delete</button>  
        </li>  
      ))}  
    </ul>  
  </div>  
}
```

2) Notes App

A notes app is a good next project because it adds editing, search, and better UI flows. Notes apps are commonly built in MERN-style tutorials and project lists.

Backend: server.js

js

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
let notes = [];
```

```
app.get("/api/notes", (req, res) => res.json(notes));
```

```
app.post("/api/notes", (req, res) => {
```

```
  const { title, content } = req.body;
```

```
  const note = { id: Date.now(), title, content };
```

```
  notes.push(note);
```

```
  res.status(201).json(note);
```

```
});
```

```
app.put("/api/notes/:id", (req, res) => {
```

```
  const id = Number(req.params.id);
```

```
  notes = notes.map(n => n.id === id ? { ...n, ...req.body } : n);
```

```
  res.json({ message: "Note updated" });
```

```
});
```

BrainStack

```
app.delete("/api/notes/:id", (req, res) => {  
  const id = Number(req.params.id);  
  notes = notes.filter(n => n.id !== id);  
  res.json({ message: "Note deleted" });  
});
```

```
app.listen(5000);
```

Frontend: App.jsx

```
jsx
```

```
import { useEffect, useState } from "react";
```

```
const API = "http://localhost:5000/api/notes";
```

```
export default function App() {  
  const [notes, setNotes] = useState([]);  
  const [title, setTitle] = useState("");  
  const [content, setContent] = useState("");  
  const fetchNotes = async () => {  
    const res = await fetch(API);  
    setNotes(await res.json());  
  };  
  useEffect(() => { fetchNotes(); }, []);  
  const addNote = async () => {  
    await fetch(API, {  
      method: "POST",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify({ title, content })  
    });  
  };
```

```
setTitle("");

setContent("");

fetchNotes();

};

const deleteNote = async (id) => {

  await fetch(`${API}/${id}`, { method: "DELETE" });

  fetchNotes();

};

return (

  <div style={{ padding: 20 }}>

    <h1>Notes App</h1>

    <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Title" />

    <br />

    <textarea value={content} onChange={(e) => setContent(e.target.value)}

placeholder="Content" />

    <br />

    <button onClick={addNote}>Save Note</button>

    {notes.map((n) => (

      <div key={n.id} style={{ border: "1px solid #ccc", marginTop: 10, padding: 10 }}>

        <h3>{n.title}</h3>

        <p>{n.content}</p>

        <button onClick={() => deleteNote(n.id)}>Delete</button>

      </div>

    ))}

  </div>

);

}
```

3) Expense Tracker

An expense tracker is useful because it teaches forms, totals, filtering, and meaningful data handling. Expense tracker apps are common in full-stack tutorials and project collections.

Backend: server.js

js

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
let expenses = [];
```

```
app.get("/api/expenses", (req, res) => res.json(expenses));
```

```
app.post("/api/expenses", (req, res) => {
```

```
  const expense = { id: Date.now(), ...req.body };
  expenses.push(expense);
  res.status(201).json(expense);
});
```

```
app.delete("/api/expenses/:id", (req, res) => {
```

```
  expenses = expenses.filter(e => e.id !== Number(req.params.id));
```

```
  res.json({ message: "Deleted" });
});
```

```
app.listen(5000);
```


Frontend: App.jsx

jsx

```
import { useEffect, useState } from "react";
```

```
const API = "http://localhost:5000/api/expenses";
```

```
export default function App() {
```

```
  const [expenses, setExpenses] = useState([]);
```

```
  const [name, setName] = useState("");
```

```
  const [amount, setAmount] = useState("");
```

```
  const load = async () => {
```

```
    const res = await fetch(API);
```

```
    setExpenses(await res.json());
```

```
  };
```

```
  useEffect(() => { load(); }, []);
```

```
  const addExpense = async () => {
```

```
    await fetch(API, {
```

```
      method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({ name, amount: Number(amount) })
```

```
    });
```

```
    setName("");
```

```
    setAmount("");
```

```
    load();
```

```
  };
```

```
  const total = expenses.reduce((sum, e) => sum + Number(e.amount), 0);
```

```
return (  
  <div style={{ padding: 20 }}>  
    <h1>Expense Tracker</h1>  
    <input value={name} onChange={(e) => setName(e.target.value)} placeholder="Expense  
name" />  
    <input value={amount} onChange={(e) => setAmount(e.target.value)}  
placeholder="Amount" />  
    <button onClick={addExpense}>Add</button>  
    <h3>Total: {total}</h3>  
  
    {expenses.map((e) => (  
      <div key={e.id}>  
        {e.name} - {e.amount}  
      </div>  
    ))}  
  </div>  
);  
}
```

4) Blog App

A blog app is a strong portfolio project because it can grow into authentication, markdown, likes, comments, and admin features. Modern full-stack blog projects often include search, drafts, and rich post management.

Backend: server.js

js

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
let posts = [
```

```
  { id: 1, title: "First Post", content: "Hello blog world!" }
];
```

```
app.get("/api/posts", (req, res) => res.json(posts));
```

```
app.post("/api/posts", (req, res) => {
```

```
  const post = { id: Date.now(), ...req.body };
  posts.push(post);
  res.status(201).json(post);
});
```

```
app.delete("/api/posts/:id", (req, res) => {
```

```
  posts = posts.filter(p => p.id !== Number(req.params.id));
```

```
  res.json({ message: "Deleted" });
});
```

```
app.listen(5000);
```

Frontend: App.jsx

jsx

```
import { useEffect, useState } from "react";
```

```
const API = "http://localhost:5000/api/posts";
```

```
export default function App() {
```

```
  const [posts, setPosts] = useState([]);
```

```
  const [title, setTitle] = useState("");
```

```
  const [content, setContent] = useState("");
```

```
  const loadPosts = async () => {
```

```
    const res = await fetch(API);
```

```
    setPosts(await res.json());
```

```
  };
```

```
  useEffect(() => { loadPosts(); }, []);
```

```
  const addPost = async () => {
```

```
    await fetch(API, {
```

```
      method: "POST",
```

```
      headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({ title, content })
```

```
    });
```

```
    setTitle("");
```

```
    setContent("");
```

```
    loadPosts();
```

```
  };
```

```
return (  
  <div style={{ padding: 20 }}>  
    <h1>Blog App</h1>  
    <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Title" />  
    <br />  
    <textarea value={content} onChange={(e) => setContent(e.target.value)}  
placeholder="Content" />  
    <br />  
    <button onClick={addPost}>Publish</button>  
  
    {posts.map((p) => (  
      <div key={p.id} style={{ border: "1px solid #ddd", marginTop: 10, padding: 10 }}>  
        <h3>{p.title}</h3>  
        <p>{p.content}</p>  
      </div>  
    ))}  
  </div>  
);  
}
```

5) Student Task Dashboard

This project is very good for your book because it feels useful and modern. It can combine tasks, deadlines, and progress tracking in one simple dashboard.

Backend: server.js

js

```
const express = require("express");
```

```
const cors = require("cors");
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
let tasks = [];
```

```
app.get("/api/tasks", (req, res) => res.json(tasks));
```

```
app.post("/api/tasks", (req, res) => {
```

```
  const task = { id: Date.now(), ...req.body }; 
```

```
  tasks.push(task);
```

```
  res.status(201).json(task);
```

```
});
```

```
app.put("/api/tasks/:id", (req, res) => {
```

```
  const id = Number(req.params.id);
```

```
  tasks = tasks.map(t => t.id === id ? { ...t, ...req.body } : t);
```

```
  res.json({ message: "Updated" });
```

```
});
```

BrainStack

```
app.delete("/api/tasks/:id", (req, res) => {  
  tasks = tasks.filter(t => t.id !== Number(req.params.id));  
  res.json({ message: "Deleted" });  
});  
  
app.listen(5000);
```

Frontend: App.jsx

```
jsx  
  
import { useEffect, useState } from "react";  
const API = "http://localhost:5000/api/tasks";  
export default function App() {  
  const [tasks, setTasks] = useState([]);  
  const [title, setTitle] = useState("");  
  
  const load = async () => {  
    const res = await fetch(API);  
    setTasks(await res.json());  
  };  
  useEffect(() => { load(); }, []);  
  const addTask = async () => {  
    await fetch(API, {  
      method: "POST",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify({ title, status: "pending" })  
    });  
    setTitle("");  
    load();  
  };  
}
```

```
const toggle = async (task) => {  
  await fetch(`${API}/${task.id}`, {  
    method: "PUT",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({ status: task.status === "done" ? "pending" : "done" })  
  });  
  load();  
};  
  
return (  
  <div style={{ padding: 20 }}>  
    <h1>Student Task Dashboard</h1>  
    <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="Task title"  
  />  
    <button onClick={addTask}>Add</button>  
    {tasks.map((task) => (  
      <div key={task.id} style={{ marginTop: 10 }}>  
        <span>{task.title} - {task.status}</span>  
        <button onClick={() => toggle(task)}>Toggle</button>  
      </div>  
    ))}  
  </div>  
);  
}
```


BrainStack

The simplest way is to deploy the **frontend on Vercel** and the **backend on Render**, then connect both with a public API URL and environment variables. This is a common modern setup for React + Node/Express projects, and it works well for small full-stack apps.

Recommended deployment setup

For your 5 projects, use this pattern:

- **UI / frontend:** Vercel.
- **Backend / API:** Render.
- **Database:** MongoDB Atlas if needed.
- **GitHub:** source code hosting and auto-deploy triggers.

This setup is especially good because Vercel handles frontend deployment very smoothly, while Render supports Express apps with simple Git-based deployment.

Step-by-step process

1. Prepare the code

Make sure each project has:

- A separate frontend and backend folder, or a clean full-stack structure.
- A working build locally.
- Environment variables moved out of the code.
- CORS enabled on the backend.

2. Push to GitHub

Upload each project to GitHub. Deployment platforms usually connect directly to your repository and redeploy automatically when you push changes.

3. Deploy the backend on Render

Create a new web service on Render and connect your backend repository. Set the start command, for example `node server.js`, and add environment variables such as database URL or API keys if you use them.

4. Deploy the frontend on Vercel

Import the frontend repo into Vercel, set the build command if needed, and update the API URL in your frontend to point to the Render backend URL. Then redeploy.

5. Connect frontend and backend

In the frontend, replace localhost API URLs with the live backend URL from Render. If you use a database, connect the backend to MongoDB Atlas or another hosted database through environment variables.

BrainStack

For your 5 projects

1. To-Do List App

- Frontend on Vercel.
- Backend on Render.
- Can use in-memory data or MongoDB for persistence.

2. Notes App

- Deploy the same way.
- Use MongoDB if you want notes to persist after restart.

3. Expense Tracker

- Backend on Render.
- Frontend on Vercel.
- Database strongly recommended if you want real tracking history.

4. Blog App

- Frontend on Vercel.
- Backend on Render.
- Database required for posts, edits, and future login features.

5. Student Task Dashboard

- Best deployed as frontend + backend split.
- Add MongoDB Atlas if you want tasks to remain saved between sessions.

Important deployment tips

- Use environment variables for API keys and database URLs.
- Enable CORS properly on the backend.
- In production, do not hardcode localhost.
- If your app uses React Router, configure fallback routing on the frontend host.
- Test each API endpoint after deployment.